

AGORA — ручные и полуавтоматические тест-кейсы пайплайна

Дата: 05.08.2026. Основано на состоянии `main` = `4c123fd`.

Этот документ описывает **что именно надо увидеть своими глазами**, чтобы считать пайплайн работающим. CDD-тесты в `evals/tests/` проверяют контракт по исходникам и на фикстурах; здесь — проверки на настоящем видео, настоящей базе и настоящей модели, то есть ровно тот уровень, который по §4 CLAUDE.md выносит вердикт по задаче.

Как читать

Колонка	Значение
Кто	 — выполняю я, результат прикладываю в PR / loop-state.  — выполняет Влад в своей среде (VPS, браузер, живая модель).  — обе стороны: я готовлю команду и фикстуру, Влад запускает.
Статус	 пройден ·  частично ·  не выполнялся ·  заблокирован незаконченной задачей

Правило честного результата. Если проверка не может выполняться (нет ключа, нет базы, нет модели) — она пишет SKIP и называет причину. Тест, который в отсутствие среды выдаёт «пройдено», хуже отсутствующего теста: он гасит подозрение вместо того, чтобы его вызывать.

0. Преднастройка

ID	Что	Кто	Статус
TC-00.1	<code>ffmpeg -version</code> и <code>ffprobe -version</code> отвечают		
TC-00.2	Фикстура на месте: <code>docs/video/15 min.mp4</code> , ~39 МБ, 917 с		
TC-00.3	Весы <code>Systran/faster-whisper-large-v3</code> скачаны (2.88 ГБ) в <code>HF_HOME</code>		

ID	Что	Кто	Статус
TC-00.4	<code>docker compose up -d</code> → все healthcheck зелёные		
TC-00.5	<code>.env.local</code> содержит ключ модели; <code>git status</code> его не показывает		

```
python3 evals/check.py
```

Ожидаемо: `secret_scan = pass`, `tasks_done` красный (16/31 — так задумано до самого конца), метрики живой среды не в состоянии `todo`.

1. Медиа-препроцессинг (задача #14)

Смысл блока: **все таймкоды в системе — таймкоды прокси-файла** (Decision Log #14). Если прокси и исходник разошлись по длительности, разъедется всё, что дальше: транскрипт, сцены, ссылки в отчёте.

ТС-1.1 — ffprobe читает реальный контейнер



```
cd services/agent-core && python3 -c "
from agent_core.media.probe import probe
i = probe('../..../docs/video/15 min.mp4')
print(i)
"
```

Ожидается: `duration_sec` ≈ 917 , ненулевые `width/height`, `fps` > 0 , `has_audio = True`. **Провал, если** длительность взята из метаданных контейнера и расходится с реальной дорожкой больше чем на 1 с.

Факт: 917.0 с, аудиодорожка найдена.

ТС-1.2 — прокси: 5 fps, ширина 640, звук отброшен



```
python3 -c "
from agent_core.media.proxy import make_proxy
```

```
from agent_core.media.probe import probe
make_proxy('../docs/video/15 min.mp4', '/tmp/proxy.mp4')
print(probe('/tmp/proxy.mp4'))
"
```

Ожидается: длительность прокси совпадает с исходником с точностью до кадра;
`has_audio = False`.

Почему звука в прокси нет и это правильно: прокси нужен для сцен и кадров, звук в нём — лишние мегабайты на каждом кадре анализа. Аудио берётся **из исходника**, а не из прокси. Это уже стоило одного ложного бага: вызов `extract_audio(proxy)` падает с «в файле нет звуковой дорожки», и это ожидаемое поведение, а не дефект.

Факт: 917.0 с, 5 fps, звука нет.

ТС-1.3 — извлечение аудиодорожки 🤖 ✅

```
python3 -c "
from agent_core.media.audio import extract_audio
from agent_core.media.probe import probe
extract_audio('../docs/video/15 min.mp4', '/tmp/audio.wav')
" && ffprobe -v error -show_entries stream=sample_rate,channels,codec_name -of
default=nw=1 /tmp/audio.wav
```

Ожидается: `codec_name=pcm_s16le`, `sample_rate=16000`, `channels=1`. 16 кГц моно — не произвольный выбор, а вход, которого ждут и Whisper, и Silero VAD; на 44.1 кГц стерео VAD даёт другие границы, и таймкоды диаризации перестают совпадать с таймкодами транскрипта.

Провал, если: частота отличается от 16000, или файл нулевой длины при `has_audio = True` в ТС-1.1.

Факт: 16 кГц, моно, 917 с.

ТС-1.4 — нарезка на фрагменты без потери суммарной длительности 🤖 ✅

```
python3 -c "
from agent_core.media.proxy import segment
from agent_core.media.probe import probe
parts = segment('/tmp/proxy.mp4', '/tmp/parts', seconds=600)
tot = 0
for p in parts:
    d = probe(p).duration_sec; tot += d; print(p.name, round(d,1))
```

```
print('сумма', round(tot,1))
"
```

Ожидается: 2 фрагмента, $614.6 + 302.4 = 917.0$ — сумма равна длительности прокси.

Первый кусок длиннее 600 с законно: `ffmpeg -f segment` режет по ключевым кадрам, а не по секундомеру.

Провал, если сумма меньше исходной длительности: значит хвост потерян, и последние минуты ролика не попадут ни в транскрипт, ни в сцены. Это самый опасный отказ в блоке, потому что внешне выглядит как успешный прогон.

Факт: $614.6 + 302.4 = 917.0$. 

ТС-1.5 — понятный отказ на битом файле

Подать на вход текстовый файл с расширением `.mp4`. **Ожидается:** `MediaError` с текстом про формат, а не `IndexError` из разбора вывода `ffprobe`. Пользователь загрузит не то, и сообщение должно объяснять, что.

2. Транскрипт и диаризация (задача #15)

ТС-2.1 — транскрипт 15-минутного ролика на `large-v3`

```
WHISPER_MODEL=large-v3 WHISPER_COMPUTE_TYPE=int8 python3 -c "  
from agent_core.asr.transcribe import transcribe  
segs = transcribe('/tmp/audio.wav')  
print(len(segs), 'сегментов')  
for s in segs[:5]: print(round(s.start,1), round(s.end,1), s.text[:70])  
print('последний конец:', round(segs[-1].end,1))  
"
```

Ожидается:

- сегментов больше нуля, текст осмысленный и на языке ролика;
- таймкоды монотонны: `start[i+1] >= start[i]`;
- `segs[-1].end` не превышает 917.0 больше чем на 0.5 с.

Провал, если конец последнего сегмента заметно больше длительности — Whisper на хвосте склонен «доезжать» за границу; в Content Pack такое подрезается (ТС-4.2), но на этом уровне расхождение надо увидеть.

Отдельно проверить галлюцинации на тишине: взять кусок без речи и убедиться, что `vad=True` даёт пустой список, а не выдуманную фразу. Именно ради этого VAD включён по умолчанию.

ТС-2.2 — large-v3 против medium по памяти

На VPS с 7.8 ГБ large-v3 уже падал по OOM, поэтому стоит `medium`. Замерить:

```
/usr/bin/time -v python3 -c "from agent_core.asr.transcribe import transcribe;
transcribe('/tmp/audio.wav')" 2>&1 | grep Maximum
```

Ожидается: пиковая память под `int8` в районе 2–3 ГБ. **Если больше 6 ГБ** — вывод не «уменьшить модель», а «инстанс меньше \$15 PRD (8 vCPU / 16 ГБ)»; это записано в блокеры отчёта.

ТС-2.3 — диаризация: границы совпадают с VAD

```
python3 -c "
from agent_core.asr.transcribe import vad_segments
from agent_core.asr.diarize import diarize
spans = vad_segments('/tmp/audio.wav')
turns = diarize('/tmp/audio.wav', spans=spans)
print(len(spans), 'участков речи;', len(turns), 'реплик;', len({t.speaker for t in
turns}), 'спикеров')
"
```

Ожидается: число спикеров правдоподобно для ролика (не 1 на диалоге и не 20 на монологе); реплики лежат внутри участков VAD. **Ожидаемый SKIP:** без `HF_TOKEN` для ruannotate — `DiarizationUnavailable`. Это законный исход, а не сбой: продукт обязан работать и без диаризации, просто без ярлыков спикеров.

ТС-2.4 — один и тот же VAD у транскрипта и диаризации (статически)

Проверяется CDD-тестом `#15`. Если границы разойдутся, спикер в Content Pack будет приписан не той реплике — дефект, который в отчёте выглядит как «персона неверно поняла героя».

3. Сцены, кадры и VLM-разбор (задача #16)

ТС-3.1 — детекция сцен на реальном ролике 🤝

```
python3 -c "  
from agent_core.frames.scenes import detect_scenes, keyframe_timestamps  
sc = detect_scenes('/tmp/proxy.mp4')  
print(len(sc), 'сцен'); print(keyframe_timestamps(sc)[:10])  
"
```

Ожидается: число сцен соответствует монтажу (для 15-минутного ролика десятки, не 2 и не 2000). Границы внутри `[0, 917]`.

Ожидаемый фолбэк: если PySceneDetect недоступен, модуль обязан вернуть равномерную нарезку по интервалу, а не пустой список. Нулевые сцены → пустой Content Pack → респонденты «смотрят» ничего.

ТС-3.2 — opencv стоит headless и импортируется 🤖 ✅

Решение: **libGL не ставится**. `opencv-python-headless` его не требует; установка libGL замаскировала бы настоящую причину отказа — pip, тихо ставящий GUI-сборку, — и притащила бы GTK/X11 в контейнер без экрана.

Вместо этого в `infra/worker.Dockerfile` стоит проверка на этапе сборки: если в образе оказалась `opencv-python` (GUI) или не оказалось ни одной сборки, сборка падает с внятным текстом.

```
docker compose build worker
```

Ожидается: сборка проходит; при подмене на GUI-сборку — падает на этом шаге, а не в рантайме на первом видео.

ТС-3.3 — дедупликация кадров 🤝

Ожидается: на статичном плане `dedupe` схлопывает почти одинаковые кадры (порог Хэмминга 4), на динамичном — почти ничего не режет. Проверять именно на двух разных кусках ролика: агрессивный порог выкидывает содержательные смены плана, слабый — раздувает счёт VLM-вызовов.

ТС-3.4 — бюджет вызовов VLM не превышаетя



Ожидается: при исчерпании `CallBudget` поднимается `CostCapExceeded`, и уже разобранные панели не теряются. Проверить, что повторный запуск на том же ролике берёт результат из кэша (`MongoCache`), а счётчик вызовов не растёт: ключ кэша считается от изображения, шаблона промпта и имени модели — смена любого из трёх обязана дать промах.

ТС-3.5 — таймкод проставлен каждой панели



Ожидается: у каждого результата анализа есть `timestamp_sec` в границах `[0, 917]`. Панель без таймкода бесполезна: отчёт не сможет сослаться на момент.

4. Склейка и Content Pack (задача #17)

ТС-4.1 — сборка пакета из трёх источников (на фикстуре)

```
cd services/agent-core && python3 -m pytest tests/test_content_pack.py -q
```

Ожидается: пакет валиден по `packages/shared/schemas/content-pack.schema.json`, поле `form = "full"`.

ТС-4.2 — таймкоды за границей ролика отбрасываются, а не молча проходят

Ожидается: сцена с `timestamp_sec` вне `[0, duration_sec]` попадает в `dropped_scenes` с причиной; хвост Whisper за границей подрезается. Отброшенное

должно быть видно — иначе разница между «в ролике нет этого момента» и «мы его потеряли» неразличима.

ТС-4.3 — компактная форма реально короче



Ожидается: `form = "compact"`, таймлайн урезан до ключевых сцен (не более 12), объём в 5–10 раз меньше полной формы.

Факт: сжатие 8.3x. Первая реализация давала 1.95x — это была не строгость порога, а дыра в реализации: таймлайн оставался полным. Чинилась реализация, не тест.

ТС-4.4 — компакт не теряет ключевую сцену



Ручная проверка на настоящем ролике: выбрать 3 момента, которые человек назвал бы ключевыми, и убедиться, что они есть в компактной форме. Отбор идёт по материалу (границы, смена настроения, плотность реплик) — на реальном контенте это надо посмотреть глазами, синтетика такое не покажет.

5. Аудитория: заземление и посевы (задачи `#5`, `#9`, `#23`, `#24`)

Самый содержательный блок. Проверяется цепочка: **корпус живых респондентов** → **критерии** → **скелет персоны** → **обогащение моделью** → **набор в БД** → **запуск**.

ТС-5.1 — генератор берёт распределения из корпуса, а не из головы

```
cd services/agent-core && python3 -m agent_core.persona.generate_cli \
  --config '{"size":20,"seed":42,"age_groups":["25-34","35-44"],"geos":
["Москва"],"use_llm":false}' \
  | python3 -m json.tool | head -40
```

Ожидается: 20 персон; возрастные группы и гео — только запрошенные; поля `big_five`, `values_and_beliefs`, `viewer_behavior` заполнены значениями, встречающимися в корпусе `data/grounding/`.

Провал, если запрошен критерий, которого в корпусе нет, и генератор молча выдал персон: он обязан отказать текстом **ValueError** с перечислением того, что в корпусе есть. Молчаливая подстановка — это выдуманная аудитория под видом заземлённой.

TC-5.2 — посев (seed) даёт воспроизводимость



Запустить TC-5.1 дважды с **seed: 42** и один раз с **seed: 43**.

Ожидается: два прогона с 42 совпадают побайтово; прогон с 43 отличается. Без этого невозможен перезапуск исследования на тех же персонах (**#30**) и невозможно отличить «модель ответила иначе» от «персоны были другие».

TC-5.3 — метрика заземления **persona_grounding** 🤝

```
python3 evals/check.py 2>&1 | grep persona_grounding
```

Ожидается: **pass** на эталонном прогоне генератора. Метрика сверяет распределения сгенерированного набора с корпусом; расхождение означает, что персоны «уехали» от живых данных.

TC-5.4 — обогащение моделью переписывает только **narrative** 🚫 ждёт ключа

Ожидается при наличии OPENAI_API_KEY:

- **meta.enriched = true, meta.llm_calls > 0;**
- **sources[i] = "model"** для обогащённых персон;
- **демография, Big Five и поведение не изменились** относительно прогона с **use_llm: false** — модель пишет портрет, а не выдумывает факты.

Ожидается без ключа: **enriched = false, degraded_reason** заполнен, **sources =** сплошь **"template"**, код возврата 0. Деградация до шаблонных портретов — рабочий результат: 20 заземлённых персон никуда не делись.

Почему sources лежит рядом с персонами, а не внутри: схема Persona DNA объявлена **additionalProperties: false**, и служебное поле внутри персоны сломало бы валидацию.

ТС-5.5 — цепочка «Аудитория» в браузере

1. Визард → шаг «Аудитория» → задать критерии → «Сгенерировать».
2. Дождаться ответа `/api/audience`.

Ожидается:

- запрос уходит на `/api/audience` (видно во вкладке Network);
- ответ содержит **настоящий** `personaSetId`, а не заглушку;
- набор появляется в разделе «Персоны» — то есть записан в Postgres, а не в память браузера;
- у каждой персоны непустое `name` (колонка `personas.name` — `NOT NULL`).

Почему это отдельный кейс: ровно здесь цепочка была разорвана в четырёх местах при зелёных CDD-тестах каждого звена. Дефект «у персоны нет имени» принадлежал промежутку между задачами `#5` и `#9` и не был виден ни одной из них. Регрессия закрыта тестом `evals/tests/test_audience_chain.py`.

ТС-5.6 — `/api/audience` не минует страницу



```
python3 evals/tests/test_audience_chain.py
```

Ожидается: 19/19 OK — проверяются все четыре звена: UI → API, API → запуск, страница `/audience`, генератор → LLM.

Известное расхождение, требует решения продукта: страница `/audience` и шаг визарда — два конструктора аудитории. Страница осталась от прототипа и хранит наборы ещё и в браузерной БД. Либо убрать, либо свести на серверное хранилище.

ТС-5.7 — портреты аудиторий (`#24`)

Ожидается: авто-дистилляция портрета из набора персон даёт связный текст, который человек узнаёт как описание именно этой аудитории, а не общие слова. Проверка глазами — метрики здесь не заменяют читателя.

6. Respondent-агенты (задача `#18`)

ТС-6.1 — структурная изоляция персон

```
cd services/agent-core && python3 -m pytest tests/test_respondent.py -q
```

Ожидается: ни один промпт не содержит данных чужой персоны — проверка идёт по **всем** отправленным промптам (20 персон × 3 репликации), а не по первым двум. Опасен не склеенный промпт, а общий изменяемый объект, который копит данные предыдущей персоны и проявляется на двадцатой во втором повторе.

Также: входные персоны не мутируются, `build_slice` — чистая функция.

ТС-6.2 — метрика `isolation_persona` на гейте

```
python3 evals/check.py 2>&1 | grep isolation_persona
```

Ожидается: при наличии `evals/artifacts/persona_answers.json` — `pass`, 0 утечек (проверено). **Без артефакта прогона** — `skip` с указанием причины, а не `pass`: метрика, зеленеющая на отсутствующих данных, отчитывалась бы об изоляции, которую никто не измерял.

ТС-6.3 — прогон не обрывается на отказе одной персоны

Ожидается: таймаут или нечитаемый JSON у одной персоны увеличивает `failures` и попадает в `failure_reasons`, но остальные ответы собираются. 19 ответов из 20 — результат; 0 ответов из-за одного таймаута — нет.

ТС-6.4 — разнообразие ответов на живой модели ждёт ключа

```
python3 evals/check.py 2>&1 | grep response_diversity
```

Ожидается: `distinct_2 >= 0.55, score_stddev >= 1.0, mode_collapse = false`.

Пороги предварительные и требуют калибровки. Процедура записана рядом с константами в `agent_core/respondent/diversity.py`: прогнать 20 персон на реальном материале, посчитать значения, затем испортить промпт (убрать из `respondent.system` пункт про право не понравиться) и посчитать снова; порог ставится между полученными значениями, ближе к нижнему.

Почему две метрики, а не одна: `distinct-2` меряет лексику и обманывается, когда персоны разными словами говорят одно и то же; дисперсия баллов меряет позицию и обманывается на случайных оценках, не связанных с текстом. Порознь каждая зеленеет на бесполезном прогоне.

Почему `temperature = 0.9`, а не `0`: при нулевой температуре метрика разнообразия мерила бы нашу конфигурацию, а не поведение модели.

ТС-6.5 — репликации 🤖✅

Ожидается: `replication_count = 3` на 20 персонах даёт 60 ответов с `replication ∈ {0,1,2}`; ответы одной персоны в разных репликациях **различаются** (иначе репликации не несут информации).

7. QA-агент (задача #19) — 🚫 не реализована

Кейсы записаны заранее, чтобы реализация делалась под них, а не наоборот.

ID	Что проверяется	Ожидается
ТС-7.1	Метрика <code>qa_catches_injected</code>	В набор ответов подсаживается заведомо негодный (противоречит материалу, вне анкеты, скопирован у соседа) — QA обязан его поймать
ТС-7.2	Ложные срабатывания	Валидные, но необычные ответы не отбраковываются: аудитория имеет право на нетипичное мнение
ТС-7.3	Отбраковка видна	Отклонённый ответ попадает в отчёт с причиной, а не исчезает

8. Analytics и отчёт (задачи #20, #21) — 🚫 не реализованы

ID	Что проверяется	Ожидается
TC-8.1	Агрегат по баллам	Средние и разброс совпадают с ручным пересчётом артефакта ответов
TC-8.2	Групповой синтез	Выводы подкреплены цитатами из верbatimов, цитаты дословны
TC-8.3	Ссылки на моменты	Каждое утверждение о сцене несёт таймкод, и таймкод попадает в ролик
TC-8.4	Отчёт в UI	Открывается, показывает сводку, верbatimы и таймкоды; пустой прогон даёт внятное «данных нет», а не пустой экран

9. Оркестратор и прогресс (задачи #13, #12) — реализованы

ID	Что проверяется	Кто	Статус	Ожидается
TC-9.1	Граф проходит все узлы			Медиа → ASR → кадры → пакет → респонденты → QA → аналитика; <code>route(short long)</code> разводит ветки, короткий режим минует <code>segment_video</code>
TC-9.2	Падение узла не рушит граф			Отказ диаризации оставляет транскрипт и попадает в <code>degraded</code> ; отметки накапливаются , а не затирают друг друга
TC-9.3	Идемпотентность			Повторный запуск с теми же параметрами не создаёт второй прогон (200 + <code>created: false</code>) и не ставит вторую задачу в очередь — прогон платный
TC-9.4	Состояние переживает перезапуск воркера			Падение на <code>pack</code> → пересоздание графа и саверта на том же хранилище → продолжение с <code>pack</code> . Узлы до него не вызываются повторно — иначе перезапуск заново гонит пятнадцатиминутную транскрипцию и оплаченные вызовы VLM
TC-9.5	SSE-прогресс			Открыть <code>/runs/<id>/progress</code> . Первым событием приходит снимок из ключа, дальше — смены узлов. Закрывать вкладку на

ID	Что проверяется	Кто	Статус	Ожидается
				середине, открыть заново: экран показывает текущий этап, а не начинается с нуля
TC-9.6	Запуск доезжает до воркера			После «Запустить» в очереди Valkey появляется задача <code>agora.run_pipeline</code> , и её id совпадает с id прогона. Расхождение не заметно сразу: каждый ретрай Celery начинал бы чистый прогон вместо продолжения с чекпоинта
TC-9.7	Отказ очереди виден на запуске			При недоступном Valkey ответ <code>/api/tasks</code> несёт <code>queueError</code> . Без этого прогон создан, воркер о нём не знает, а экран показывает QUEUED вечно — это выглядит как медленная система, а не как недоехавшая задача
TC-9.8	Прогресс чужого прогона не отдаётся			<code>GET /api/tasks/<чужой id>/progress</code> → 404. Открытый поток на чужой идентификатор сам по себе сообщал бы, что такой прогон существует

Заметка про сверку протокола. Заголовки сообщения Celery собираются в `apps/web/lib/server/queue.ts` вручную — клиента Celery для Node нет, а веб и воркер общаются только через Valkey. Имена заголовков не выписаны по памяти: CDD-тест [#12](#) сверяет их с тем, что выдаёт сам Celery (`as_task_v2`). Обновление Celery, переименовавшее заголовки, покраснит тест, а не сломает прод молча. **Оболочка сообщения (base64-тело, `delivery_info`) на живом брокере не проверялась** — это оставшийся риск блока, снимается пунктом TC-9.6.

10. Сквозной прогон (задача [#22](#)) — не начата

ID	Что	Ожидается
TC-10.1	<code>e2e_short</code>	Короткий ролик проходит от загрузки до отчёта без ручного вмешательства
TC-10.2	<code>e2e_long</code>	Фикстура готова: <code>docs/video/15 min.mp4</code> , 917 с. Проходит целиком; время прогона зафиксировано

ID	Что	Ожидается
TC-10.3	Golden-сеты ×3	Три прогона на одном материале с одним seed дают сопоставимые выводы; расхождение в выводах при совпадающих персонах — сигнал нестабильности

10-бис. Документация API (OpenAPI 3.1)

ID	Что	Кто	Статус	Ожидается
TC-10b.1	Спецификация валидна			<code>openapi-spec-validator</code> принимает <code>packages/shared/openapi/agora.openapi.json</code> как OpenAPI 3.1
TC-10b.2	Спецификация не отстала от кода			<code>evals/tests/test_openapi_coverage.py</code> — 31 путь, 42 операции; сверка идёт в обе стороны . Пропущенный маршрут — невидимая часть API; лишний путь хуже: по нему напишут вызов, получают 404 и будут искать ошибку у себя
TC-10b.3	Swagger UI рендерится			<code>/api</code> показывает заголовок, OAS 3.1 , выбор сервера, Authorize и все 12 разделов операций
TC-10b.4	Документация закрыта сессией			Без входа <code>/api</code> → 307 на <code>/login</code> , <code>/api/openapi</code> → 401. Проверено на dev-сервере
TC-10b.5	«Try it out» ходит с кукой			Выполнить GET /api/personas из Swagger UI под своей сессией → 200, а не 401
TC-10b.6	Публичные маршруты совпадают с кодом			В спецификации security: [] стоит ровно у тех операций, чьи пути перечислены в PUBLIC_PATHS

Решение, которое осталось за вами. Страница `/api` намеренно **не** добавлена в **PUBLIC_PATHS**: она перечисляет параметры и коды ответов всех защищённых маршрутов, то есть готовую карту поверхности атаки. Открывать её наружу — решение продукта, а не побочный эффект публикации.

Если открывать — **не** добавляйте `"api": isPublicPath` сравнивает по префиксу со слэшем, и такая запись откроет заодно всё дерево `/api/*`, то есть весь API без сессии. Нужен отдельный публичный путь вида `/api-docs`.

11. Безопасность и мультиарендность (сквозной блок)

ID	Что	Кто	Ожидается
TC-11.1	<code>rls_tenant</code>		Арендатор не видит чужих персон, наборов, прогонов. Проверять из-под <code>agora_app</code> и из-под владельца таблиц — <code>FORCE ROW LEVEL SECURITY</code> стоит на всех 16 таблицах
TC-11.2	<code>FORCE</code> не снят		<code>SELECT relname, relforcerowsecurity FROM pg_class</code> — ни одного <code>false</code> среди таблиц арендатора
TC-11.3	Нет <code>BYPASSRLS</code>		<code>migrate.sh</code> останавливается, если находит у роли <code>SUPERUSER</code> или <code>BYPASSRLS</code>
TC-11.4	<code>secret_scan</code>		✅ <code>pass</code> . Проверяет и <code>evals/tests</code> — раньше пропускал пароли там
TC-11.5	Артефакты прогона не в git		✅ <code>evals/artifacts/</code> закрыт. Закоммиченный артефакт делает метрику ответом на чужой давний запуск
TC-11.6	Ротация паролей		 <code>owner@agora.local</code> , <code>member@agora.local</code> в истории git; плюс root VPS, <code>gen_user</code> , <code>agora_login</code>

Сводка покрытия

Блок	Задачи	Пройдено	Ждёт среды/ключа	Заблокировано
Медиа	#14	4	1	—
ASR	#15	1	3	—
Кадры/VLM	#16	1	4	—
Content Pack	#17	3	1	—
Аудитория	#5 #9 #23 #24	4	2	1 (ключ)
Респонденты	#18	4	—	1 (ключ)
QA	#19	—	—	3

Блок	Задачи	Пройдено	Ждёт среды/ ключа	Заблокировано
Аналитика/отчёт	#20 #21	—	—	4
Оркестратор/ прогресс	#13 #12	3	5	—
E2E	#22	—	—	3
Безопасность	сквозной	2	4	—

Итого: 22 пройдено, 20 ждут среды, 12 заблокированы незаконченными задачами.